



南開大學  
Nankai University

南 開 大 學

自然语言处理实验报告

---

作业一：文本分类实现

---

李浩天

学号：2110133

专业：物联网工程

# 目录

|                             |          |
|-----------------------------|----------|
| <b>一、 实验概述</b>              | <b>1</b> |
| <b>二、 数据准备与实验协议</b>         | <b>1</b> |
| (一) 数据来源与字段                 | 1        |
| (二) 训练集、验证集和测试集划分           | 1        |
| (三) 评价指标                    | 2        |
| <b>三、 任务一：基于词袋的文本分类</b>     | <b>2</b> |
| (一) 词袋表示的基本思想               | 2        |
| (二) Binary Bag of Words     | 3        |
| (三) Word Frequency          | 3        |
| (四) Logistic Regression 分类器 | 3        |
| (五) Logistic Regression 求解器 | 4        |
| (六) 结果                      | 4        |
| <b>四、 任务二：基于词向量的文本分类</b>    | <b>5</b> |
| (一) 词向量与文档向量                | 5        |
| (二) 预训练 GloVe               | 5        |
| (三) 在 AG News 上训练 Word2Vec  | 5        |
| (四) 在 NYT 训练集上训练 Word2Vec   | 6        |
| (五) 三组词向量实验的共同后续流程          | 6        |
| (六) 结果                      | 6        |
| <b>五、 任务三：BERT 预训练模型微调</b>  | <b>6</b> |
| (一) BERT 表示与分类原理            | 6        |
| (二) 数据转换与批量分词               | 7        |
| (三) 模型训练、选择与测试              | 7        |
| (四) BERT 学习率实验结果            | 7        |
| <b>六、 实验结果与分析</b>           | <b>8</b> |
| <b>七、 总结</b>                | <b>8</b> |

## 一、实验概述

本实验以 New York Times (NYT) 新闻的八分类任务为对象, 比较三类文本表示方法。Task1 使用词袋表示, Task2 使用词向量平均表示, Task3 微调预训练 BERT 模型。词袋实验分别使用 Binary BoW 和 Word Frequency 构造文档向量; 词向量实验使用预训练 GloVe、在 AG News 上训练的 Word2Vec, 以及在 NYT 训练集上训练的 Word2Vec; BERT 实验则对预训练模型针对 NYT 数据集微调。

各实验使用同一份 NYT 训练集、验证集和测试集。传统表示方法均以 Logistic Regression 作为分类器, BERT 使用模型自带的分类层。验证集用于观察训练效果及选择模型, 最终性能统一在测试集上报告 Accuracy 和 Macro-F1。

本次实验的数据集, 预训练词向量, 预训练模型来源:

- NYT 新闻数据集: [https://huggingface.co/datasets/dstefa/New\\_York\\_Times\\_Topics](https://huggingface.co/datasets/dstefa/New_York_Times_Topics)
- AG News 数据集: [https://huggingface.co/datasets/fancyzhx/ag\\_news](https://huggingface.co/datasets/fancyzhx/ag_news)
- 预训练 GloVe 词向量: <https://nlp.stanford.edu/data/glove.6B.zip>
- BERT 预训练模型: <https://huggingface.co/google-bert/bert-base-uncased>

NYT 原始语料来源于《纽约时报》发表的新闻文章。新闻文本本身并未按本实验的八类主题任务提供统一的分类标签, 因而不能直接作为本实验所需的 `text-label` 数据使用。本实验在 Hugging Face 上找到了基于这些新闻整理的主题分类数据集; 其预处理文件已提供新闻文本 `text`、主题名称 `topic_name` 和主题编号 `topic_id`。后续将 `topic_id` 转换为整数标签 `label`, 得到实验所需的两列数据。

代码流程可以概括为: 下载原始数据, 预处理并划分 NYT 数据集, 构造各方法所需的文本表示, 训练分类器, 最后在验证集和测试集上计算评价指标。对应的主要脚本为 `dataset.py`、`prep_data.py`、两个 `task01` 脚本、`task02word2vec.py` 和 `task03.py`。

## 二、数据准备与实验协议

### (一) 数据来源与字段

`dataset.py` 使用 Hugging Face Hub 的 `snapshot_download` 下载 AG News 和 NYT 数据。NYT 原始文件为 `nyt-preprocessed_news_256000.csv`, 后续实验使用其中的新闻文本和主题编号。AG News 在本实验中只作为一种 Word2Vec 词向量的训练语料; AG News 的类别标签不参与 NYT 分类器训练。

`prep_data.py` 读取 NYT 原始文件后, 先对 `text` 执行 `str.strip()`, 去除文本首尾空白; 再将 `topic_id` 转为整数, 命名为 `label`; 最后只保留 `text` 与 `label` 两列。类别编号为 0 至 7, 分别对应体育、文化娱乐、商业金融、健康、生活时尚、科学技术、政治和犯罪主题。

需要区分这里的文本整理与分词: `prep_data.py` 仅整理字段及首尾空白, 并没有把新闻预先转换为词序列。词袋方法、Word2Vec 方法和 BERT 方法各自在后续脚本中执行与模型相适应的分词。

### (二) 训练集、验证集和测试集划分

数据划分使用 `StratifiedGroupKFold(n_splits=10, shuffle=True, random_state=42)`。其中, `y=df["label"]` 指定分层依据为类别标签, `groups=df["text"]` 指定分组依据为完整文

本。脚本遍历十个折，并将每个样本所属的折号保存到 `fold_id`。折号 0 构成测试集，折号 1 构成验证集，其余八折构成训练集，随后分别写入 `train.csv`、`val.csv` 和 `test.csv`。

分层的作用是尽量维持各折的类别比例；分组的作用是让相同文本落在同一折，避免重复新闻同时出现在训练集与测试集，从而使测试成绩偏高。分层分组折的实际样本数仍应以划分后的文件为准。本次生成的文件恰好满足 80% : 10% : 10%，具体数量如表 1 所示。检查三个文件中的完整文本，也没有发现跨集合重复。

表 1: NYT 数据集划分

| 数据集 | 样本数    | 占比  | 每类样本数 |
|-----|--------|-----|-------|
| 训练集 | 204800 | 80% | 25600 |
| 验证集 | 25600  | 10% | 3200  |
| 测试集 | 25600  | 10% | 3200  |

所有模型读取上述三个 CSV 文件，保证比较时使用相同的目标任务与数据划分。词袋词表只由 NYT 训练集建立；在 NYT 上训练的 Word2Vec 也只读取 NYT 训练集文本。验证集和测试集不参与这些表示的拟合。

### (三) 评价指标

设测试集共有  $N$  个样本，真实标签为  $y_i$ ，预测标签为  $\hat{y}_i$ ，则分类准确率定义为

$$\text{Accuracy} = \frac{1}{N} \sum_{i=1}^N \mathbf{1}(\hat{y}_i = y_i), \quad (1)$$

其中  $\mathbf{1}(\cdot)$  为指示函数。Accuracy 衡量总体预测正确的比例，但单独使用它不能展示各类别之间的差异。

对于第  $k$  类，记真正例、假正例和假负例分别为  $TP_k$ 、 $FP_k$  和  $FN_k$ ，则

$$P_k = \frac{TP_k}{TP_k + FP_k}, \quad (2)$$

$$R_k = \frac{TP_k}{TP_k + FN_k}, \quad (3)$$

$$F_{1,k} = \frac{2P_k R_k}{P_k + R_k}. \quad (4)$$

本实验共有  $K = 8$  类，Macro-F1 为

$$\text{Macro-F1} = \frac{1}{K} \sum_{k=1}^K F_{1,k}. \quad (5)$$

它先分别评价每个类别，再对类别等权平均，因此不会让样本较多的类别在最终分数中占更大权重。虽然本次划分的八个类别数量相同，同时报告两个指标仍有助于检查总体正确率与各类表现是否一致。代码分别调用 `accuracy_score` 和设置 `average="macro"` 的 `f1_score` 实现计算。

## 三、任务一：基于词袋的文本分类

### (一) 词袋表示的基本思想

词袋模型先从训练语料建立词表  $V$ ，再按照词表中每个词的固定位置构造文档向量。若词表大小为  $|V|$ ，每篇文档就对应一个  $|V|$  维向量。该表示保留词项的出现信息，却不保留词序和句

法关系。例如，两个句子包含相同的词而排列顺序不同，可能得到完全相同的词袋向量。

两个词袋脚本都使用 `nltk.word_tokenize` 分词，并传入 `preserve_line=True`。随后通过 `any(char.isalnum() for char in token)` 保留至少包含一个字母或数字的词元，过滤纯标点词元。这里的规则是“至少含一个字母或数字”，并不等同于只允许字母或数字。代码没有显式执行词干提取、词形还原或停用词删除。

`CountVectorizer` 接收上述自定义分词函数。`token_pattern=None` 表示不再使用其默认的正则词元模式，`lowercase=True` 则使大小写统一处理。执行 `fit_transform(train["text"])` 时，它依据训练文本建立词表并转换训练集；之后的 `transform` 只沿用已有词表转换验证集和测试集。后两组数据中未进入训练词表的词，不会产生新的特征列。这一流程既保持三个特征矩阵的列含义一致，也避免利用测试集内容建立词表。

词袋维度较高，而单篇新闻只包含词表中的少量词。因此，`CountVectorizer` 返回稀疏矩阵，只存储非零元素及其位置，无须为每篇文档显式保存大量零值。本次两个词袋脚本建立的词表大小均为 123944，训练集矩阵形状均为  $204800 \times 123944$ 。

## (二) Binary Bag of Words

在 `task01binary_bag_of_words.py` 中，`CountVectorizer` 设置 `binary=True`。对于文档  $d$  和词表中的词  $v_j$ ，特征定义为

$$x_j(d) = \begin{cases} 1, & v_j \text{ 在 } d \text{ 中至少出现一次,} \\ 0, & v_j \text{ 未在 } d \text{ 中出现.} \end{cases} \quad (6)$$

因此，同一词在文档中出现一次或多次，均对应数值 1。这种表示强调词项是否提供了主题线索，而不让重复次数直接放大该词的特征值。

脚本先读取三个数据文件，调用训练集上的 `fit_transform` 建立词表与训练特征，再依次转换验证集和测试集。随后以训练特征和训练标签拟合 Logistic Regression，并分别输出验证集、测试集的 Accuracy 和 Macro-F1。

## (三) Word Frequency

在 `task01word_frequency.py` 中，流程与 Binary BoW 相同，区别是 `binary=False`。此时第  $j$  维记录词  $v_j$  在当前文档中的出现次数：

$$x_j(d) = \text{count}(v_j, d). \quad (7)$$

例如文档包含词序列 “apple apple banana”，相应的两个词频特征分别为 2 和 1。词频表示保留了重复信息；当同一主题词在一篇文章中多次出现时，其数值会增大。相应地，文章长度和词的重复方式也会影响特征值。本实现没有额外进行基于文本长度的归一化，因此结果为原始词频表示的效果。

## (四) Logistic Regression 分类器

两个词袋脚本均使用 `LogisticRegression`，设置 `solver` 时测试了多个求解器，其他参数固定 `max_iter=1000`，`random_state=42`。对于输入特征  $x$ ，多分类线性模型为每个类别计算一个得分

$$z_k = w_k^T x + b_k. \quad (8)$$

分类时选择得分最高的类别；若将得分转换为概率，可使用 softmax：

$$p(y = k | \mathbf{x}) = \frac{\exp(z_k)}{\sum_{j=1}^K \exp(z_j)}. \quad (9)$$

模型通过训练数据调整权重，使正确类别获得更高的预测概率。相同的分类器配置用于两种词袋特征，使本组比较主要反映输入表示的差异。脚本在训练后直接调用 `predict` 获取类别，再计算两项指标。

## (五) Logistic Regression 求解器

Logistic Regression 的求解器负责寻找使训练目标最小的模型参数。更换求解器并不改变词袋特征或八分类任务，但不同算法的迭代方式、内存开销、收敛速度及所支持的正则化形式不同。scikit-learn 的 `LogisticRegression` 提供以下六种求解器：

- `lbfgs`：拟牛顿方法，用有限的历史梯度信息近似二阶信息，是较稳健的默认选择；支持本实验使用的多分类模型与  $L_2$  正则化。
- `newton-cg`：利用共轭梯度法求解牛顿方向，适用于多分类及  $L_2$  正则化。
- `newton-cholesky`：显式计算 Hessian 矩阵并求解线性方程组；当样本数远大于特征数时可能合适，但 Hessian 的内存开销随特征数平方增长，不适合本实验的大词表。
- `liblinear`：基于 LIBLINEAR，适合部分较小规模的问题，也支持  $L_1$  正则化；它不能直接优化完整的多分类目标，处理本实验的八分类任务需要额外采用 One-vs-Rest 方案。
- `sag`：随机平均梯度方法，每次迭代利用部分样本的信息更新参数，适合较大的数据集；其快速收敛依赖特征具有相近的尺度。
- `saga`：在 `sag` 基础上扩展，适合大规模稀疏数据和多分类任务；除  $L_2$  外，还支持  $L_1$  及 Elastic-Net 正则化，同样需要关注特征尺度。

## (六) 结果

两组实验使用相同的数据、词袋表示及 Logistic Regression 配置，仅改变 `solver`。得到以下结果：两种求解器的分类指标非常接近。SAGA 在两种表示下的测试 Macro-F1 都略高，但在

表 2: NYT 数据集上两种词袋表示的求解器比较

| 表示方法           | 求解器    | 训练/s  | 预测/ms | 迭代次数 | 验证 Acc | 验证 Macro-F1 | 测试 Acc | 测试 Macro-F1 |
|----------------|--------|-------|-------|------|--------|-------------|--------|-------------|
| Binary BoW     | L-BFGS | 30.35 | 5.06  | 219  | 0.8905 | 0.8904      | 0.8873 | 0.8872      |
| Binary BoW     | SAGA   | 27.48 | 5.17  | 125  | 0.8903 | 0.8902      | 0.8879 | 0.8878      |
| Word Frequency | L-BFGS | 42.74 | 5.17  | 313  | 0.8887 | 0.8886      | 0.8866 | 0.8865      |
| Word Frequency | SAGA   | 80.98 | 5.22  | 369  | 0.8893 | 0.8891      | 0.8870 | 0.8869      |

Binary BoW 上训练稍快，在词频表示上却明显更慢。因此不能笼统说某个求解器更快。L-BFGS 的迭代与 SAGA 的数据遍历含义不同，表中的迭代次数也不能直接相除比较效率。SAGA 对特征尺度较敏感，词频特征的尺度差异可能是它在这里收敛较慢的原因之一。Newton-Cholesky 的内存：词表有 123,944 个词、8 个类别；连同截距，参数维度是  $8(123944 + 1) = 991560$ 。该求解器需要构造约  $991560 \times 991560$  的 Hessian。若按 float64 每个元素 8 字节计算，约需 7.15 TiB，试运行时的报错也计算出了内存开销。稀疏的输入矩阵也无法消除这个 Hessian 的平方级内存开销，因此它不适合这次高维 BoW 实验。

## 四、任务二：基于词向量的文本分类

### (一) 词向量与文档向量

词袋表示为不同词分配不同的坐标，但坐标本身不表达词与词之间的相似性。词向量将一个词映射到低维连续空间中的向量；训练目标促使具有相关上下文或共现关系的词形成有用的几何表示。本任务统一使用 100 维词向量。

分类器需要每篇文档对应一个固定长度的输入。`task02word2vec.py` 对文档分词后，只保留当前词向量表中能找到的词。若有效词元依次为  $w_1, \dots, w_m$ ，其文档向量为

$$e(d) = \frac{1}{m} \sum_{i=1}^m e(w_i), \quad m > 0. \quad (10)$$

这里对词元而非去重后的词集合求平均，所以重复出现的词会被重复计入。若一篇文档没有任何可用词向量，代码保留预先初始化的 100 维零向量，同时把它计入 `empty_count`。词向量平均提供了简单且固定维度的文档表示，但会丢失词序，也无法明确区分否定关系和不同词在句中的作用。

代码使用 `word_tokenize(text.lower(), preserve_line=True)`，再过滤纯标点词元。与词袋脚本相比，这里在调用分词函数前就将文本转为小写，以便匹配词向量表中的词。生成的训练、验证和测试特征矩阵分别为  $204800 \times 100$ 、 $25600 \times 100$  和  $25600 \times 100$ 。

### (二) 预训练 GloVe

GloVe 利用语料中的词语共现统计学习词向量。它与下面的 Word2Vec 实验在向量训练语料和学习方法上不同，但本实验将两者转换为文档向量时采用相同的逐词平均方法。

当 `METHOD="glove"` 时，`load_glove` 从 `glove.6B.zip` 中直接打开 `glove.6B.100d.txt`。文件每行应包含一个词和 100 个数值；代码仅处理拆分后恰有 101 项的行，并将数值转换为 `float32` 数组，存入以词为键的字典。这样无需先将整个压缩包解压到另一目录。运行记录显示，最终加载的词向量表包含 400000 个词。

该词向量表不根据 NYT 测试集训练。NYT 文档只通过查表和求平均得到特征；之后再使用 NYT 训练集标签拟合 Logistic Regression。对于在 GloVe 表中找不到的词，代码直接跳过，而没有构造未知词向量。

### (三) 在 AG News 上训练 Word2Vec

当 `METHOD="ag"` 时，脚本从 AG News 的训练集 Parquet 文件读取 `text` 列，逐篇分词后交给 `gensim.models.Word2Vec`。Parquet 是面向列存储的数据格式；这里指定 `columns=["text"]`，因此只读取训练词向量所需的文本列。AG News 的标签没有被读取，也没有用于 NYT 分类。

Word2Vec 根据词的上下文学习表示。本实现设置 `sg=0`，选择 CBOW (Continuous Bag of Words) 形式，即利用窗口内的上下文词预测中心词；`window=5` 控制上下文窗口，`vector_size=100` 指定向量维度，`min_count=2` 排除训练语料中出现次数少于两次的词，`epochs=5` 指定词向量训练轮数。`workers=1` 和 `seed=42` 用于控制训练配置。代码没有显式设置的 Word2Vec 参数由所用 `gensim` 版本的默认值决定。

训练完成后，脚本取 `model.wv` 作为词到向量的查询对象，将 NYT 各集合中的新闻转换为平均词向量，再仅用 NYT 训练集拟合分类器。运行记录中的 AG News 词向量表大小为 57176。由于词向量训练语料与分类语料来源不同，NYT 中的部分词可能在该词向量表中不存在；代码通过成员判断跳过这些词。

#### (四) 在 NYT 训练集上训练 Word2Vec

当 `METHOD="nyt"` 时, `train_word2vec(train_tokens)` 仅使用 NYT 训练集文本学习词向量, 其余 Word2Vec 配置、文档向量计算方式和分类器训练流程与 AG News 分支相同。运行记录中的词向量表大小为 69873。

这组实验使词向量训练语料与目标分类数据处于同一新闻来源, 但不能由此单独断定“语料来源”是性能差异的唯一原因: 两份语料的规模、词汇覆盖和内容分布也不同。验证集与测试集只使用训练得到的词向量进行转换, 不参与 NYT Word2Vec 的拟合。

#### (五) 三组词向量实验的共同后续流程

无论 `METHOD` 选择 `glove`、`ag` 还是 `nyt`, 代码最终都调用同一 `make_document_vectors` 函数生成三组文档矩阵, 统计没有可用词向量的文档数, 并使用相同配置的 Logistic Regression 训练与评价。运行记录中, AG News、NYT 和 GloVe 分支在训练集、验证集、测试集上的零向量文档数分别为 (3, 2, 2)、(0, 0, 1) 和 (1, 0, 0)。

最初使用 SAGA 求解逻辑回归时, AG News 组在 56 轮后停止, 但 NYT 和 GloVe 组达到 `max_iter=1000` 后仍出现 `ConvergenceWarning`。这表示求解器在规定的迭代上限内未满足停止条件, 并不表示分类程序运行失败。提高迭代上限至 3000 带来了较长的训练时间, 因此最终统一改用 L-BFGS。三组 L-BFGS 训练均在 122 至 139 次迭代后停止, 拟合时间约为 4 至 5 秒。

#### (六) 结果

三种方法均将词向量取平均, 得到 100 维文档向量, 再使用逻辑回归进行 NYT 文本分类。表 3 展示了统一使用 L-BFGS 求解器时的结果。

表 3: 不同词向量来源的 NYT 文本分类结果

| 词向量来源            | 词表大小    | 拟合时间/s | 迭代次数 | 验证 Acc | 验证 Macro-F1 | 测试 Acc | 测试 Macro-F1   |
|------------------|---------|--------|------|--------|-------------|--------|---------------|
| AG News Word2Vec | 57,176  | 4.15   | 122  | 0.7528 | 0.7518      | 0.7511 | 0.7499        |
| NYT Word2Vec     | 69,873  | 4.25   | 126  | 0.8435 | 0.8433      | 0.8434 | <b>0.8431</b> |
| GloVe 6B         | 400,000 | 4.95   | 139  | 0.8201 | 0.8196      | 0.8216 | 0.8210        |

NYT 语料训练的 Word2Vec 取得最高的测试 Macro-F1 (0.8431), 分别比 GloVe 和 AG News 语料训练的 Word2Vec 高约 2.21 和 9.32 个百分点。NYT 词向量与下游分类数据来自同一语料, 这可能有助于表示任务相关的词汇和语义。GloVe 的词表最大, 但分类结果并非最好, 说明仅凭词表大小不能判断文档表示的效果。

### 五、任务三：BERT 预训练模型微调

#### (一) BERT 表示与分类原理

BERT 与前述“先生成固定文档向量, 再训练独立线性分类器”的流程不同。它先将文本变成子词序列, 经过多层 Transformer 编码, 再由序列分类层输出八个类别的得分。微调时, 分类层与预训练编码器一起依据 NYT 标签更新参数。

Transformer 的自注意力机制使一个位置的表示能够结合序列中其他位置的信息。其基本计算形式为

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V, \quad (11)$$

其中  $Q$ 、 $K$ 、 $V$  分别为查询、键和值矩阵， $d_k$  是键向量维度。与词袋和简单词向量平均相比，这种表示能够随上下文改变词元表示，并保留更多顺序与上下文信息。

本实验调用 `google-bert/bert-base-uncased`。模型配置文件显示其编码器包含 12 层、12 个注意力头，隐藏层维度为 768；序列分类头输出 8 个类别。这里的“uncased”对应不区分大小写的预训练模型，实际词元切分规则由与该模型配套的 tokenizer 决定，不能与 NLTK 的词级分词直接等同。

## (二) 数据转换与批量分词

`task03.py` 首先检查 PyTorch 是否可以使用 CUDA；若不可用，脚本直接抛出异常。接着，`read_split` 使用 `pandas.read_csv` 读取 `text` 和 `label`，将 `label` 重命名为 `labels`，并通过 `Dataset.from_pandas(..., preserve_index=False)` 转为 Hugging Face Dataset。统一的 `labels` 字段供后续模型和 Trainer 读取监督标签。

`AutoTokenizer.from_pretrained` 加载与预训练模型对应的 tokenizer。`tokenize_batch` 对一个批次的文本执行分词，设置 `truncation=True` 和 `max_length=64`。因此，超过长度上限的输入会被截断；这一长度限制也意味着较长新闻靠后的内容可能无法进入模型。三个数据集分别通过 `Dataset.map(..., batched=True)` 批量完成转换，并用 `remove_columns=["text"]` 移除原始字符串列。

代码在分词时没有要求将全部样本预先补齐到固定长度。`DataCollatorWithPadding(tokenizer)` 在组成训练或评价批次时，根据当前批次的长度动态补齐，并生成模型所需的批量输入。动态补齐可以避免每篇短新闻都提前补到长度上限。`attention_mask` 用于区分实际词元位置与补齐位置。

## (三) 模型训练、选择与测试

使用 `AutoModelForSequenceClassification.from_pretrained` 加载 `google-bert/bert-base-uncased`，并设置 `num_labels=8`。文本经对应 tokenizer 编码，最大长度为 64 个 token，批内使用动态补齐。为固定新建分类层的随机初始化，在创建模型前调用 `set_seed(42)`，同时在 `TrainingArguments` 中设置 `seed=42`。

模型训练 3 个 epoch，权重衰减为 0.01；若当前 CUDA 设备支持 BF16，则使用 BF16。初步比较训练批量后，后续学习率实验将每设备训练批量和评价批量均固定为 64。在相同数据划分、批量和随机种子下，比较了  $1 \times 10^{-5}$ 、 $2 \times 10^{-5}$ 、 $3 \times 10^{-5}$ 、 $4 \times 10^{-5}$ 、 $6 \times 10^{-5}$  和  $8 \times 10^{-5}$  六个学习率，学习率使用默认的线性调度。

## (四) BERT 学习率实验结果

早期尝试了每设备训练批量 8、32 和 64。批量增大后运行耗时明显缩短，但前两组运行尚未在模型创建前显式固定随机种子，因此不将它们之间的微小分数差异严格归因于批量大小。随后固定批量为 64、随机种子为 42，进行学习率对照。表 4 列出每组按验证 Macro-F1 选出的结果；六组最佳检查点均位于第三轮。

验证 Macro-F1 随学习率增大先升后出现波动，本次最高值出现在  $4 \times 10^{-5}$ 。验证 loss 与 Macro-F1 的排序并不完全一致：loss 还反映预测概率的置信程度，而 Macro-F1 根据最终类别计算。因此本实验遵循预先设定的验证 Macro-F1 选择规则。

表 4: 批量为 64 时的 BERT 学习率对照

| 学习率                | 验证 Macro-F1     | 验证 loss  |
|--------------------|-----------------|----------|
| $1 \times 10^{-5}$ | 0.917633        | 0.248114 |
| $2 \times 10^{-5}$ | 0.922815        | 0.236774 |
| $3 \times 10^{-5}$ | 0.923232        | 0.242166 |
| $4 \times 10^{-5}$ | <b>0.923707</b> | 0.251445 |
| $6 \times 10^{-5}$ | 0.921938        | 0.265865 |
| $8 \times 10^{-5}$ | 0.922976        | 0.271566 |

## 六、 实验结果与分析

表 5 汇总目前各方法在同一 NYT 测试集上的结果。其中 LR 表示 Logistic Regression。Task 1 对每种词袋表示选用验证集 Macro-F1 较高的求解器；Task 2 三种词向量表示统一使用 L-BFGS；Task 3 使用验证集选出的批量 64、学习率  $4 \times 10^{-5}$  配置。

表 5: NYT 测试集上的阶段性结果

| 任务     | 文本表示                | 分类模型        | Accuracy | Macro-F1 |
|--------|---------------------|-------------|----------|----------|
| Task 1 | Binary BoW          | LR (L-BFGS) | 0.887305 | 0.887228 |
| Task 1 | Word Frequency      | LR (SAGA)   | 0.886992 | 0.886933 |
| Task 2 | AG News Word2Vec 平均 | LR (L-BFGS) | 0.751133 | 0.749926 |
| Task 2 | NYT Word2Vec 平均     | LR (L-BFGS) | 0.843359 | 0.843092 |
| Task 2 | GloVe 平均            | LR (L-BFGS) | 0.821563 | 0.820955 |
| Task 3 | BERT 微调             | BERT 分类层    | 0.922852 | 0.922702 |

两种词袋表示的测试分数接近，本次 Binary BoW 的 Macro-F1 略高于 Word Frequency。三种平均词向量方法的分数均低于词袋方法。将一篇文档的词向量直接平均会得到紧凑的稠密表示，但可能削弱少数关键主题词的作用，而词袋与词频方法得到的文本向量保留了所有在词表内关键词的信息，可能对分类任务更优；

平均词向量方法中，NYT 训练集上训练的 Word2Vec 表现最好，AG News Word2Vec 表现最低，预训练 GloVe 位于两者之间。NYT 词向量的训练语料与分类与验证数据的来源一致，可能有利于学习任务相关词汇的表示；不同语料的规模、内容和词汇分布也会共同影响结果。GloVe 虽有更大的词表，却没有取得最高分类分数。

BERT 的程序输出在目前各方法中最高。它使用预训练编码器，并在分类训练中继续更新编码器参数；与前两类表示相比，模型能够根据上下文生成表示。不过，各方法还存在模型结构和预训练来源等差异，不能把全部分数差异归因于是否保留词序。

## 七、 总结

本实验在统一的 NYT 数据划分上，实现了二值词袋、词频词袋、三种来源的平均词向量表示，以及 BERT 微调。词袋方法形成高维稀疏特征；平均词向量形成 100 维稠密特征；BERT 在分类任务中继续更新预训练编码器。当前结果显示，文本表示和训练设置都会影响分类表现，BERT 的程序输出最高。